Case Study 1: University Scholarship Allocation

1. Problem Statement

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Marks are out of 100; each subject >= 50, average >= 70, attendance >= 75%, income <= Rs. 300000. Rank by average descending, then attendance descending, then income ascending, then name.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

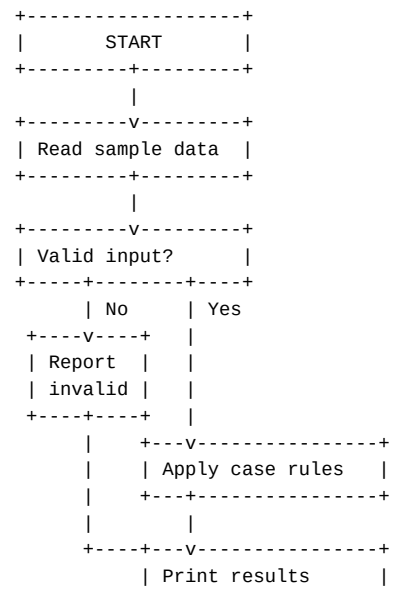
Compare, classify, rank, or summarize records as required.

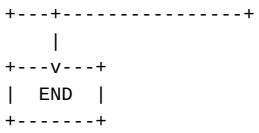
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
students = [
    ("Asha", [85, 90, 80], 90, 200000),
    ("Bala", [78, 82, 80], 85, 150000),
    ("Charan", [92, 45, 88], 95, 100000),
    ("Divya", [85, 90, 80], 95, 180000)
]
eligible = []
for name, marks, attendance, income in students:
    avg = sum(marks) / len(marks)
    if min(marks) >= 50 and avg >= 70 and attendance >= 75 and income <= 300000:
        eligible.append((name, avg, attendance, income))
# Sort using simple selection sort and the stated tie rules.
for i in range(len(eligible)):
    best = i
    for j in range(i + 1, len(eligible)):
        a, b = eligible[j], eligible[best]
        if (-a[1], -a[2], a[3], a[0]) < (-b[1], -b[2], b[3], b[0]):
            best = j
    eligible[i], eligible[best] = eligible[best], eligible[i]
for rank, row in enumerate(eligible, 1):
    print(rank, row[0], "Average:", round(row[1], 2))
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
1 Divya Average: 85.0
2 Asha Average: 85.0
3 Bala Average: 80.0
```

10. Conclusion

The university scholarship allocation case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Energy slabs: first 100 units Rs. 3, next 100 Rs. 5, above 200 Rs. 8 per unit; fixed Rs. 50; 5% surcharge when units > 300; 10% rebate when units <= 100. Increase > 30% is unusual.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

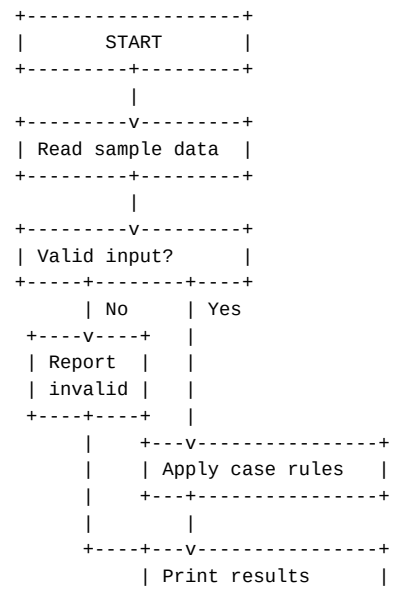
Compare, classify, rank, or summarize records as required.

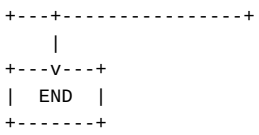
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
months = [("Jan", 90), ("Feb", 180), ("Mar", 350)]
previous = None
for month, units in months:
    if units < 0:
        print(month, "Invalid units")
        continue
    energy = min(units, 100) * 3
    energy += max(0, min(units - 100, 100)) * 5
    energy += max(0, units - 200) * 8
    amount = energy + 50
    if units > 300:
        amount *= 1.05
    if units <= 100:
        amount *= 0.90
    unusual = previous is not None and amount > previous * 1.30
    print(month, "Rs.", round(amount, 2), "Unusual:", unusual)
    previous = amount
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Jan Rs. 288.0 Unusual: False
Feb Rs. 750 Unusual: True
Mar Rs. 2152.5 Unusual: True
```

10. Conclusion

The progressive electricity billing case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 3: Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

All charges must be nonnegative. Senior patients (age >= 60) get 15% concession; others with insurance get 10%; no combined concession. Final bill < Rs. 2000 is LOW, <= Rs. 10000 MEDIUM, otherwise HIGH.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

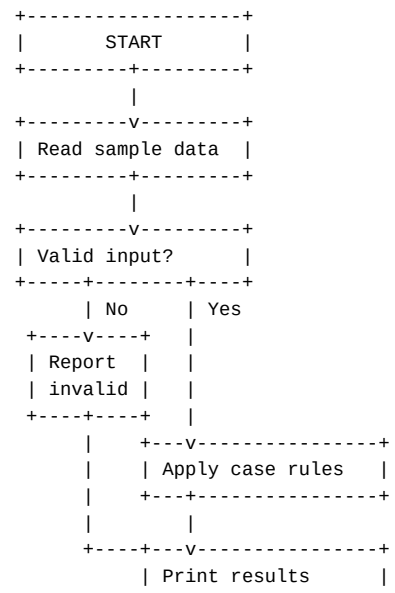
Compare, classify, rank, or summarize records as required.

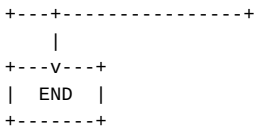
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
patients = [
    ("Ravi", 65, False, 500, 1500, 4000),
    ("Sita", 35, True, 300, 500, 1000),
    ("Kiran", 25, False, 500, 2000, 9000)
]
for name, age, insured, consult, diagnostic, treatment in patients:
    if age < 0 or min(consult, diagnostic, treatment) < 0:
        print(name, "Invalid record")
        continue
    total = consult + diagnostic + treatment
    rate = 0.15 if age >= 60 else (0.10 if insured else 0)
    bill = total * (1 - rate)
    category = "LOW" if bill < 2000 else ("MEDIUM" if bill <= 10000 else "HIGH")
    print(name, "Bill:", round(bill, 2), "Class:", category)
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Ravi Bill: 5100.0 Class: MEDIUM
Sita Bill: 1620.0 Class: LOW
Kiran Bill: 11500 Class: HIGH
```

10. Conclusion

The hospital billing and patient classification case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

HRA = 20% and DA = 10% of basic; overtime Rs. 200/hour. Deduction is 5% on first Rs. 30000 of gross and 10% on remainder. Nonnegative inputs.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

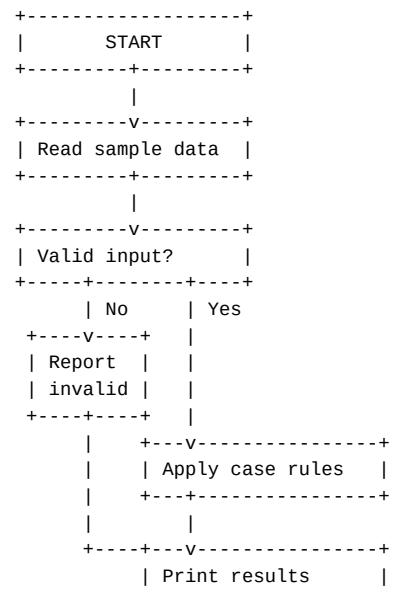
Compare, classify, rank, or summarize records as required.

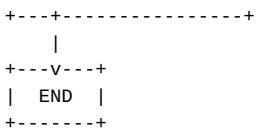
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
employees = [("Anu", 25000, 5, 1000), ("Binu", 40000, 10, 2000)]
for name, basic, hours, incentive in employees:
    if min(basic, hours, incentive) < 0:
        print(name, "Invalid input")
        continue
    hra = basic * 0.20
    da = basic * 0.10
    overtime = hours * 200
    gross = basic + hra + da + overtime + incentive
    deduction = min(gross, 30000) * 0.05 + max(0, gross - 30000) * 0.10
    net = gross - deduction
    print(name, "Gross:", round(gross, 2), "Deduction:", round(deduction, 2), "Net:",
round(net, 2))
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Anu Gross: 34500.0 Deduction: 1950.0 Net: 32550.0
Binu Gross: 56000.0 Deduction: 4100.0 Net: 51900.0
```

10. Conclusion

The employee payroll and deduction analysis case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 5: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Plans: Basic Rs. 199 for 2 GB/100 minutes, Plus Rs. 349 for 5 GB/300 minutes, Pro Rs. 599 for 10 GB/1000 minutes; all 28 days. Extra GB Rs. 50, extra minute Rs. 1. Usage and budget nonnegative.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

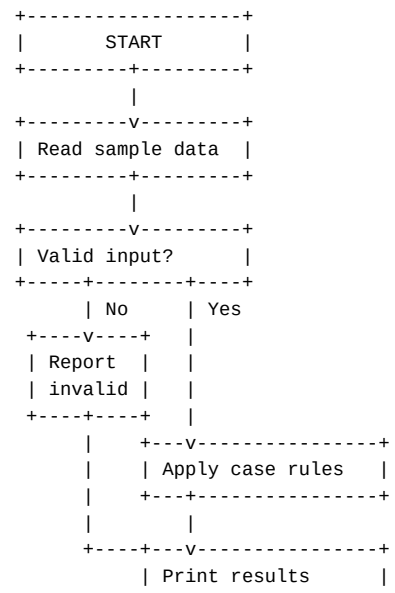
Compare, classify, rank, or summarize records as required.

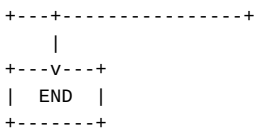
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
plans = [("Basic", 199, 2, 100, 28), ("Plus", 349, 5, 300, 28),
         ("Pro", 599, 10, 1000, 28)]
used_gb, used_minutes, days, budget = 6, 250, 28, 700
choices = []
if min(used_gb, used_minutes, days, budget) < 0:
    print("Invalid usage")
else:
    for name, fee, gb, minutes, validity in plans:
        if days <= validity:
            cost = fee + max(0, used_gb - gb) * 50
            cost += max(0, used_minutes - minutes) * 1
            print(name, "Cost:", cost)
            if cost <= budget:
                choices.append((cost, name))
    if choices:
        print("Best:", min(choices)[1], "Rs.", min(choices)[0])
    else:
        print("No plan fits the budget")
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Basic Cost: 549
Plus Cost: 399
Pro Cost: 599
Best: Plus Rs. 399
```

10. Conclusion

The multi-plan telecommunications cost optimization case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 6: Railway Fare and Reservation Calculator

1. Problem Statement

Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Base rate per km: first 100 km Rs. 1/km, additional distance Rs. 0.75/km. Sleeper multiplier 1, AC multiplier 2. Children under 12 get 50% off base fare; seniors >= 60 get 30% off. Reservation fee Rs. 20/passenger; cancellation penalty 25% of ticket price.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

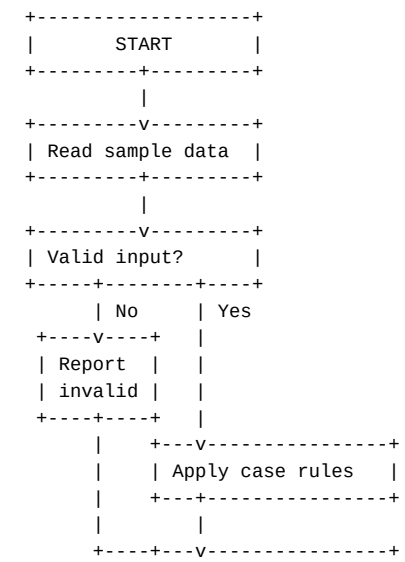
Compare, classify, rank, or summarize records as required.

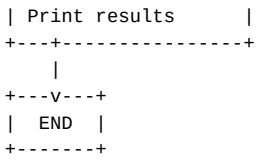
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
passengers = [("Anu", 25, "Sleeper"), ("Bala", 65, "AC"),
              ("Chintu", 9, "Sleeper")]
distance = 150
if distance <= 0:
    print("Invalid distance")
else:
    base = min(distance, 100) * 1 + max(0, distance - 100) * 0.75
    total = 0
    for name, age, travel_class in passengers:
        if age < 0 or travel_class not in ("Sleeper", "AC"):
            print(name, "Invalid passenger")
            continue
        fare = base * (2 if travel_class == "AC" else 1)
        if age < 12:
            fare *= 0.50
        elif age >= 60:
            fare *= 0.70
        fare += 20
        total += fare
        print(name, "Fare:", round(fare, 2), "Cancel penalty:", round(fare * 0.25, 2))
    print("Total:", round(total, 2))
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Anu Fare: 157.5 Cancel penalty: 39.38
Bala Fare: 212.5 Cancel penalty: 53.12
Chintu Fare: 88.75 Cancel penalty: 22.19
Total: 458.75
```

10. Conclusion

The railway fare and reservation calculator case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 7: Retail Billing with Multiple Discount Rules

1. Problem Statement

Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Products have a category tax: FOOD 5%, CLOTHES 12%, OTHER 18%. Discount 10% on pretax subtotal >= Rs. 2000, capped at Rs. 500. Nonnegative prices and positive integer quantities.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

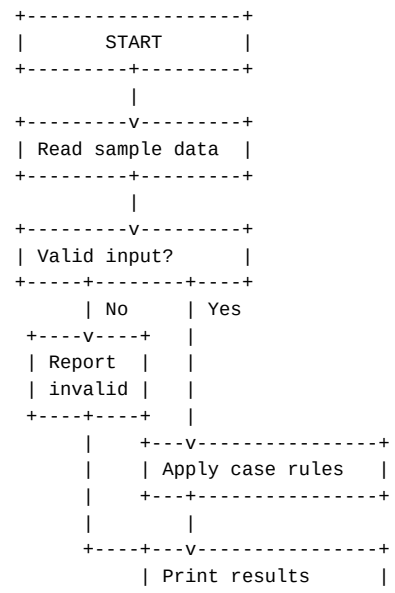
Compare, classify, rank, or summarize records as required.

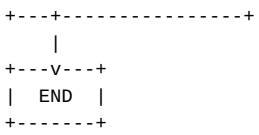
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
items = [("Rice", "FOOD", 500, 2), ("Shirt", "CLOTHES", 800, 2),
         ("Lamp", "OTHER", 300, 1)]
subtotal = 0
for name, category, price, qty in items:
    if price < 0 or qty <= 0 or category not in ("FOOD", "CLOTHES", "OTHER"):
        print(name, "Invalid item")
        continue
    subtotal += price * qty
discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
print("Subtotal:", round(subtotal, 2), "Discount:", round(discount, 2))
total = 0
for name, category, price, qty in items:
    if price < 0 or qty <= 0 or category not in ("FOOD", "CLOTHES", "OTHER"):
        continue
    line = price * qty
    allocated_discount = discount * line / subtotal if subtotal else 0
    rate = 0.05 if category == "FOOD" else (0.12 if category == "CLOTHES" else 0.18)
    amount = (line - allocated_discount) * (1 + rate)
    total += amount
    print(name, "Total with tax:", round(amount, 2))
print("Invoice total:", round(total, 2))
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Subtotal: 2900 Discount: 290.0
Rice Total with tax: 945.0
Shirt Total with tax: 1612.8
Lamp Total with tax: 318.6
Invoice total: 2876.4
```

10. Conclusion

The retail billing with multiple discount rules case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 8: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Wastage is a percentage of material cost, not units; all expenses nonnegative. Profit target is 20% of total cost. Selling price is per unit and all produced units sell.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

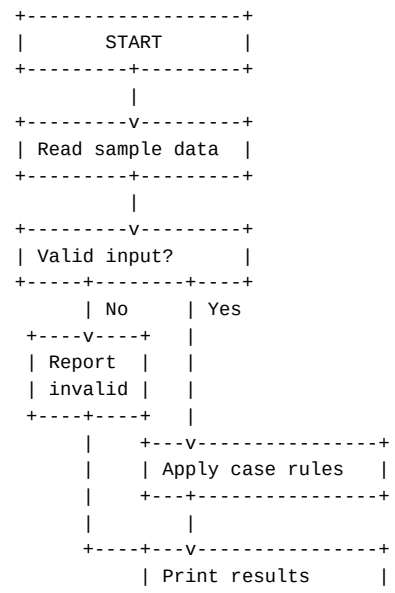
Compare, classify, rank, or summarize records as required.

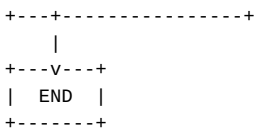
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
units = 100
material, labour, machine = 10000, 5000, 2000
waste_percent = 5
selling_price = 220
if units <= 0 or min(material, labour, machine, waste_percent, selling_price) < 0:
    print("Invalid production data")
else:
    waste = material * waste_percent / 100
    cost = material + labour + machine + waste
    revenue = units * selling_price
    profit = revenue - cost
    target = cost * 0.20
    print("Cost:", round(cost, 2))
    print("Revenue:", round(revenue, 2))
    print("Profit:", round(profit, 2))
    print("Target met:", profit >= target)
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Cost: 17500.0
Revenue: 22000
Profit: 4500.0
Target met: True
```

10. Conclusion

The manufacturing cost and profitability analysis case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Exchange rates are INR per unit of foreign currency. Conversion fee 2% of converted value. Budget and expenses nonnegative. Largest category is determined by INR total including fees.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

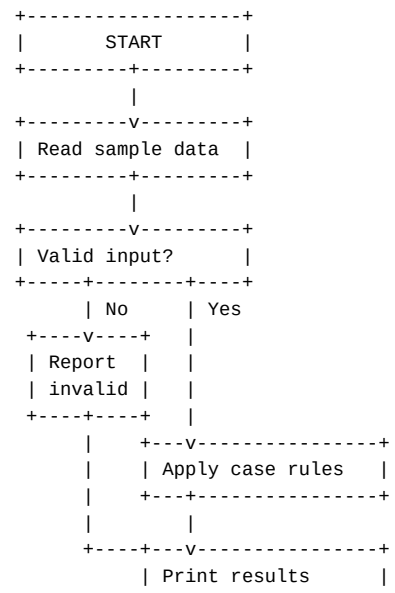
Compare, classify, rank, or summarize records as required.

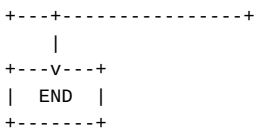
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
expenses = [("Hotel", 200, 85), ("Food", 150, 85),
            ("Transport", 100, 85)]
budget_inr = 45000
fee_rate = 0.02
total = 0
largest = ("", -1)
for category, foreign_amount, inr_rate in expenses:
    if foreign_amount < 0 or inr_rate <= 0:
        print(category, "Invalid expense")
        continue
    inr = foreign_amount * inr_rate * (1 + fee_rate)
    total += inr
    print(category, "INR:", round(inr, 2))
    if inr > largest[1]:
        largest = (category, inr)
print("Total INR:", round(total, 2))
print("Within budget:", total <= budget_inr)
print("Largest category:", largest[0])
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Hotel INR: 17340.0
Food INR: 13005.0
Transport INR: 8670.0
Total INR: 39015.0
Within budget: True
Largest category: Hotel
```

10. Conclusion

The multi-currency travel budget analysis case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.

Case Study 10: Campus Event Financial Planning

1. Problem Statement

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2. Objectives

Develop a simple Python solution that processes the given records, applies the specified decision rules, validates inputs, and prints understandable results.

3. Assumptions (illustrative rules)

Fixed costs: venue, equipment, transport. Variable costs: catering and certificates per attendee. Sponsorship and registration fee are nonnegative. Break-even count assumes every registrant attends and variable cost per person stays constant.

The source assignment does not provide numerical rates or sample records; these values are example assumptions for a working solution and may be changed to match faculty instructions.

4. Algorithm

Start and initialize sample input records.

Check required inputs for invalid values.

Use loops and conditional statements to calculate results under the stated rules.

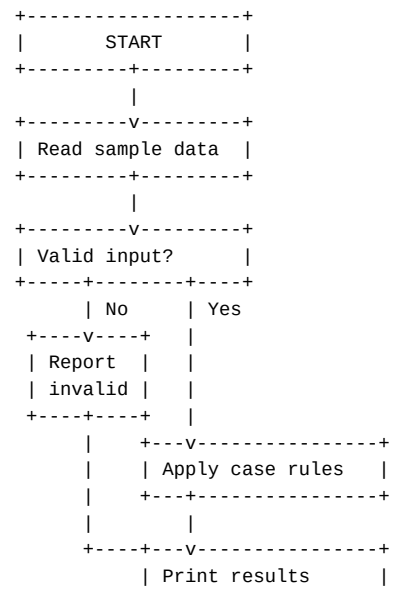
Compare, classify, rank, or summarize records as required.

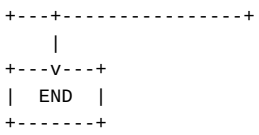
Display the results and stop.

5. Pseudocode

```
START
  READ sample data and policy values
  FOR each record
    IF record invalid THEN report error
    ELSE apply policy calculations and conditions
  END FOR
  COMPARE / RANK / SUMMARIZE where required
  PRINT results
STOP
```

6. Flowchart





7. Python Source Code

```
venue, equipment, transport = 10000, 5000, 3000
catering_per_person, certificate_per_person = 150, 20
attendees, registration_fee, sponsor = 100, 300, 5000
if min(venue, equipment, transport, catering_per_person,
       certificate_per_person, attendees, registration_fee, sponsor) < 0:
    print("Invalid event inputs")
else:
    fixed = venue + equipment + transport
    variable_per_person = catering_per_person + certificate_per_person
    cost = fixed + attendees * variable_per_person
    collections = attendees * registration_fee
    shortfall = max(0, cost - collections - sponsor)
    print("Total cost:", cost)
    print("Registration collections:", collections)
    print("Sponsorship shortfall:", shortfall)
    margin = registration_fee - variable_per_person
    if margin <= 0:
        print("Break-even impossible with current fee")
    else:
        needed = max(0, fixed - sponsor)
        minimum = int(needed // margin)
        if minimum * margin < needed:
            minimum += 1
        print("Minimum registrations:", minimum)
```

8. Test Cases

Test 1: Run the included sample data; verify the printed values against the policy. Test 2: Change one input to a boundary value (such as zero, threshold, or limit). Test 3: Enter an invalid negative input and verify that the program reports or skips invalid data where applicable.

9. Sample Output (captured from running the code)

```
Total cost: 35000
Registration collections: 30000
Sponsorship shortfall: 0
Minimum registrations: 100
```

10. Conclusion

The campus event financial planning case study demonstrates input handling, calculations, conditional logic, loops, and clear reporting using beginner-level Python.